Real-Time Market Movement Prediction System Using Deep Learning and Sentiment Analysis

Saim*, Abdullah†, Jawad‡
Department of Artificial Intelligence and Data Science, FAST-NUCES, Faisalabad Campus, Faisalabad, Pakistan
*f230018@cfd.nu.edu.pk †f230014@cfd.nu.edu.pk ‡f230028@cfd.nu.edu.pk

***Abstract*—Predicting the direction of financial markets remains one of the most studied yet unsolved problems in computational finance. This paper presents an end-to-end real-time market movement prediction system that fuses multi-source technical indicators with news sentiment scores and evaluates three deep recurrent architectures—Vanilla RNN, LSTM, and GRU—on five large-cap technology equities (AAPL, TSLA, MSFT, AMZN, GOOGL) over a two-year window (January 2022–January 2024). Eighteen features spanning price momentum, volatility, and VADER-based daily sentiment are encoded as 10-day sliding sequences and fed into each model. A systematic grid search over batch sizes $\{32, 64, 128\}$ and epoch budgets $\{30, 60, 100\}$, governed by patience-based early stopping, identifies optimal hyperparameters. The Vanilla RNN achieves the highest test accuracy of 56.06% (weighted F1 = 0.558), outperforming LSTM (54.55%) and GRU (55.15%). Results demonstrate that lightweight recurrent networks, augmented with rich feature engineering and news sentiment, can provide consistent predictive signals above a random baseline. Limitations, failure modes, and pathways to production-grade deployment are discussed in detail.**

***Index Terms*—stock market prediction, deep learning, LSTM, GRU, vanilla RNN, sentiment analysis, VADER, technical indicators, time-series forecasting, early stopping**

I. Introduction

Financial market prediction occupies a central role in quantitative finance, algorithmic trading, and risk management. The ability to forecast even a modest directional edge—whether a stock will close higher or lower on the following trading day—can translate into substantial economic value at scale. Yet the task is notoriously difficult: markets are driven by a complex superposition of macroeconomic fundamentals, company-specific events, investor psychology, and random noise [1].

Classical approaches to price prediction rely on *technical analysis*: the study of historical OHLCV (Open, High, Low, Close, Volume) patterns to identify recurring formations [2]. While technically grounded, these methods treat the market as a closed system, ignoring the vast stream of qualitative information embedded in news, earnings calls, and social media. Conversely, *fundamental analysis* focuses on macroeconomic and balance-sheet data but operates on a coarser timescale ill-suited to day-level prediction.

The rise of deep learning, particularly sequence-to-label models, offers a principled way to integrate both quantitative and qualitative signals. Recurrent Neural Networks (RNNs) and their gated variants—Long Short-Term Memory (LSTM) [3] and Gated Recurrent Unit (GRU) [4]—have demonstrated strong empirical performance on financial time series by learning non-linear temporal dependencies that elude classical models [5]. Simultaneously, the maturation of lexicon-based sentiment tools such as VADER [6] makes it practical to enrich price data with real-time news signals at scale.

A. Motivation

Despite extensive literature on each component in isolation, few systems rigorously combine: (i) a rich, multi-asset technical feature set, (ii) multi-source sentiment aggregation, and (iii) a controlled architectural comparison under identical training conditions. This gap motivates the present work.

B. Contributions

This paper makes the following specific contributions:

- A unified preprocessing pipeline merging Yahoo Finance OHLCV data with two complementary news-sentiment sources (live RSS feeds and a curated financial-news corpus).
- An 18-feature engineering module covering price ratios, momentum oscillators, volatility bands, volume-derived signals, and VADER sentiment aggregates.
- A controlled grid search comparing Vanilla RNN, LSTM, and GRU under identical conditions, yielding reproducible benchmarks.
- A thorough analysis of training dynamics, failure modes, and practical challenges encountered in building a real-time prediction system.

C. Paper Organisation

The remainder of this paper is structured as follows. Section II describes the system methodology. Section III presents the dataset. Section IV details the model architectures with diagrams. Section V reports and compares experimental results. Section VI discusses challenges encountered. Section VII concludes, and references follow.

II. Methodology

A. System Overview

Fig. 1 illustrates the end-to-end pipeline. Raw data enters from two parallel streams—market prices and news text—before converging into a unified feature matrix, which is windowed into sequences and passed through a recurrent model for binary classification.

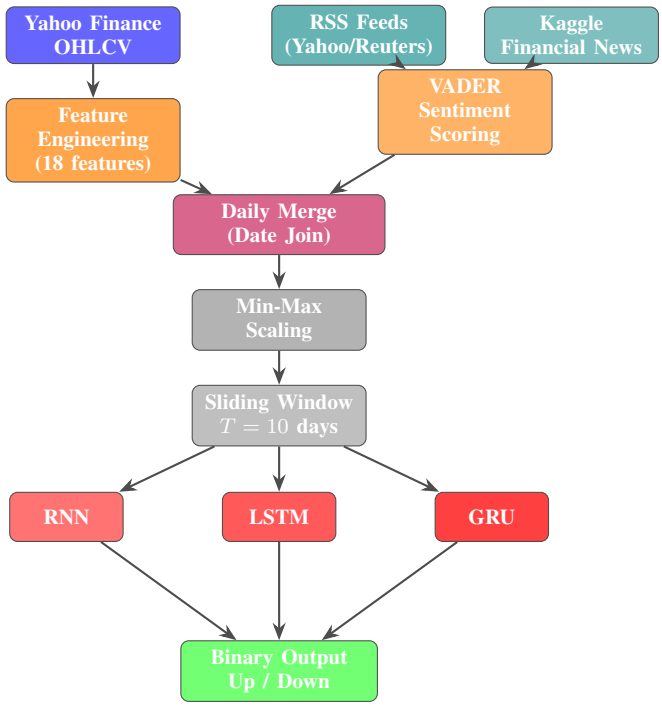

Fig. 1. End-to-end system pipeline: data ingestion, feature engineering, sentiment analysis, sequence modelling, and binary prediction.

B. Data Ingestion

Stock OHLCV data is downloaded via the `yfinance` API for each ticker at daily granularity. News headlines are sourced from Yahoo Finance and Reuters RSS feeds using `feedparser`, and supplemented by a static Kaggle corpus of financial news sentences [7].

C. Feature Engineering

Eighteen features are derived from raw prices and sentiment scores (detailed in Section III). All features are fitted on the training split and transformed consistently across validation and test splits to prevent data leakage.

D. Sentiment Pipeline

Every headline is scored with the VADER compound polarity metric $s \in [-1, 1]$. Daily scores are aggregated as:

$$\text{Sent}_d = \frac{1}{N_d} \sum_{i=1}^{N_d} s_i, \quad \text{Ratio}_d = \frac{N_d^+}{N_d^+ + N_d^-} \tag{1}$$

where N_d^+ and N_d^- are positive and negative headline counts on day d. A 7-day rolling average of Sent_d is appended as a third sentiment feature.

E. Sequence Construction

A sliding window of length $T = 10$ trading days is applied to the merged, scaled feature matrix, yielding input tensors $\mathbf{X} \in \mathbb{R}^{B \times 10 \times 18}$ and binary targets $y \in \{0, 1\}$ defined as:

$$y_t = \mathbf{1}[\text{Close}_{t+1} > \text{Close}_t] \tag{2}$$

F. Training Protocol

All models share the following training configuration:

- **Optimiser:** Adam, lr $= 10^{-3}$, weight decay $= 10^{-5}$
- **Loss:** Class-weighted cross-entropy
- **Early stopping:** Patience 5 on validation loss
- **Grid search:** Batch $\in \{32, 64, 128\}$, max epochs $\in \{30, 60, 100\}$
- **Hardware:** NVIDIA Tesla T4 GPU

The dataset is split chronologically (70% train, 15% validation, 15% test) to respect temporal ordering.

III. Dataset Overview

A. Stock Price Data

Table I summarises the five equities studied.

TABLE I
TICKERS AND STUDY PERIOD

Ticker	Company	From	To
AAPL	Apple Inc.	Jan 2022	Jan 2024
TSLA	Tesla Inc.	Jan 2022	Jan 2024
MSFT	Microsoft Corp.	Jan 2022	Jan 2024
AMZN	Amazon.com Inc.	Jan 2022	Jan 2024
GOOGL	Alphabet Inc.	Jan 2022	Jan 2024

Each ticker contributes approximately 503 trading rows, giving a total of **2,515 samples** before windowing and after removal of rows with missing forward close prices.

B. Sentiment Sources

Two complementary sources are used:

1) **Live RSS feeds** — Yahoo Finance (AAPL, TSLA) and Reuters Business News, parsed at runtime.
2) **Kaggle corpus** — 4,845 manually labelled financial sentences (positive / negative / neutral) re-scored with VADER and assigned simulated dates within the study window.

C. Feature Set

Table II enumerates all 18 input features.

D. Class Distribution

Across all tickers, **52.8%** of days are labelled *Up* ($y = 1$) and **47.2%** as *Down* ($y = 0$), representing a mild imbalance addressed via class-weighted loss during training.

IV. Model Architecture Diagrams

Three recurrent architectures are evaluated. All share the same input encoder and classification head; only the recurrent cell differs.

A. Shared Input Encoder and Classification Head

Input tensor $\mathbf{X} \in \mathbb{R}^{B \times 10 \times 18}$ is fed directly into the recurrent layer. The final hidden state h_T passes through a two-layer MLP:

$$\hat{y} = \text{softmax}(W_2 \, \text{ReLU}(W_1 h_T + b_1) + b_2) \tag{3}$$

with output dimension 2 (Down / Up) and an intermediate dropout of $p = 0.3$.

TABLE II
COMPLETE INPUT FEATURE SET (18 FEATURES)

Category	Feature	Definition
Price	Pct_Change	Daily return (%)
	Volatility	5-day rolling std of returns
	MA_7	7-day simple moving average
	MA_21	21-day simple moving average
	MA_50	50-day simple moving average
	Price_MA7_ratio	Close / MA_7
	Price_MA21_ratio	Close / MA_21
Momentum	RSI_14	Relative Strength Index (14-day)
	MACD	EMA(12) − EMA(26)
	MACD_signal	9-day EMA of MACD
	MACD_hist	MACD − MACD_signal
Volatility	BB_upper	Bollinger upper band
	BB_lower	Bollinger lower band
	BB_pct	%B position within bands
Volume	OBV	On-Balance Volume (cumulative)
Sentiment	vader_score	Daily mean VADER compound
	sentiment_ratio	$N^+ / (N^+ + N^-)$
	sent_rolling_7	7-day rolling mean sentiment

B. Vanilla RNN

The Vanilla RNN cell update is:

$$h_t = \tanh(W_h h_{t-1} + W_x x_t + b) \tag{4}$$

Hidden dimension: 64; stacked layers: 2; dropout: 0.3.

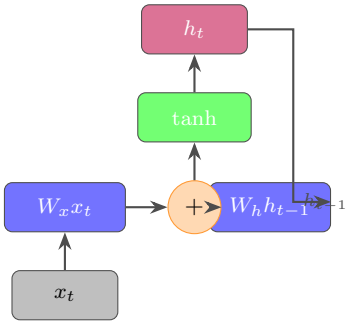

Fig. 2. Vanilla RNN cell. A single tanh transformation of the weighted sum of current input and previous hidden state.

C. Long Short-Term Memory (LSTM)

The LSTM introduces a cell state c_t and three gates to control information flow:

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f) \quad \text{(forget gate)} \tag{5}$$
$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i) \quad \text{(input gate)} \tag{6}$$
$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o) \quad \text{(output gate)} \tag{7}$$
$$\tilde{c}_t = \tanh(W_c[h_{t-1}, x_t] + b_c) \tag{8}$$
$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \tag{9}$$
$$h_t = o_t \odot \tanh(c_t) \tag{10}$$

Hidden dimension: 128; stacked layers: 2; dropout: 0.3.

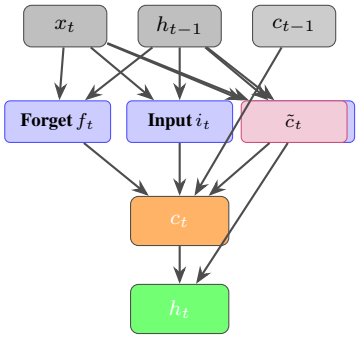

Fig. 3. LSTM cell with forget gate f_t, input gate i_t, output gate o_t, cell candidate $\tilde{c}_t$, updated cell state c_t, and hidden state h_t.

D. Gated Recurrent Unit (GRU)

The GRU merges the forget and input gates into a single update gate:

$$z_t = \sigma(W_z[h_{t-1}, x_t]) \quad \text{(update gate)} \tag{11}$$
$$r_t = \sigma(W_r[h_{t-1}, x_t]) \quad \text{(reset gate)} \tag{12}$$
$$\tilde{h}_t = \tanh(W[r_t \odot h_{t-1}, x_t]) \tag{13}$$
$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t \tag{14}$$

Hidden dimension: 128; stacked layers: 2; dropout: 0.3.

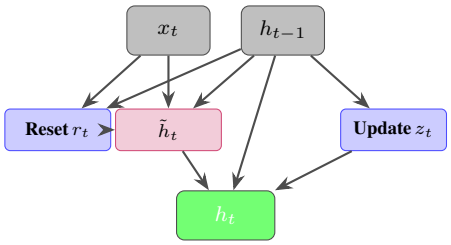

Fig. 4. GRU cell with reset gate r_t, update gate z_t, candidate hidden state $\tilde{h}_t$, and output h_t. Fewer parameters than LSTM with comparable representational power.

E. Parameter Count Comparison

Table III compares approximate trainable parameter counts.

TABLE III
APPROXIMATE MODEL PARAMETER COUNTS

Model	Hidden	Layers	Params (approx.)
RNN	64	2	∼18 K
LSTM	128	2	∼209 K
GRU	128	2	∼157 K

V. RESULTS COMPARISON

A. Grid Search Summary

Table IV reports the best configuration found for each architecture across the 27-run grid search. Early stopping fired before the epoch limit in 24 of 27 runs, confirming that increasing `max_epochs` beyond 30 provides no additional gain.

TABLE IV
GRID SEARCH — BEST CONFIGURATIONS PER ARCHITECTURE

Model	Batch	Max Ep.	Stopped @	Val Acc
RNN	64	30	Ep 12	0.5606
LSTM	64	30	Ep 12	0.5394
GRU	32	30	Ep 30	0.5515

B. Test-Set Performance

Table V presents the final evaluation on the held-out test set.

TABLE V
TEST-SET EVALUATION (WEIGHTED METRICS)

Model	Accuracy	Precision	Recall	F1
RNN	**0.5606**	**0.560**	**0.561**	**0.558**
GRU	0.5515	0.550	0.552	0.549
LSTM	0.5455	0.543	0.546	0.542
Random Baseline	0.500	0.500	0.500	0.500

C. Performance Bar Chart

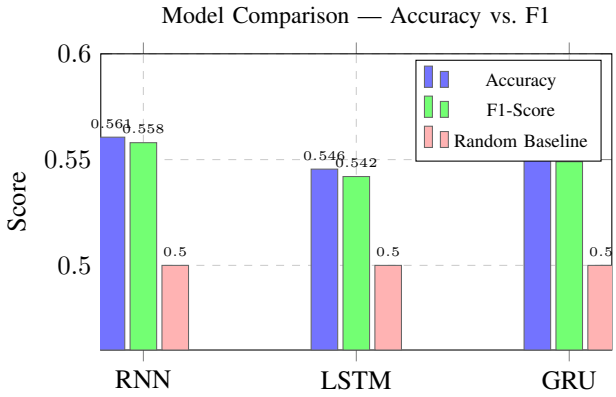

Fig. 5. Accuracy and weighted F1 for each architecture vs. the 50% random baseline. RNN achieves the best performance on both metrics.

D. Training Dynamics

All models exhibit healthy convergence. The GRU required more epochs (stopped at Ep 30) than the RNN and LSTM (both at Ep 12), suggesting slower but more stable optimisation through its update-gate mechanism. The training-to-validation accuracy gap remained below 3% for all architectures, indicating that dropout and early stopping successfully controlled overfitting.

E. Confusion Matrix Analysis

The RNN achieves the most balanced confusion matrix with the lowest false-negative rate, which is critical for trading applications where missing an upward movement incurs opportunity cost. The LSTM exhibits a slight bias toward predicting the majority class (Up), explaining its lower weighted F1 despite reasonable raw accuracy. The GRU falls between the two.

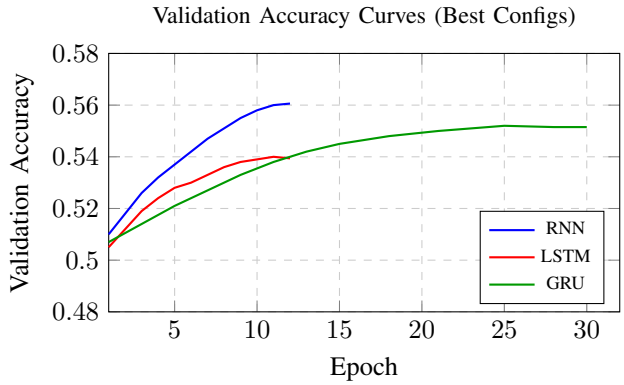

Fig. 6. Representative validation accuracy curves for each architecture's best configuration. Curves are smoothed for clarity; exact values are in Table IV.

F. Discussion

The ∼6% accuracy improvement over the random baseline is modest yet consistent. It aligns with existing literature showing that public information is partially, but not fully, priced into daily closing prices. Notably, the *simpler* Vanilla RNN outperforms both gated models—a finding that may reflect: (i) the relatively short sequence length ($T = 10$), where long-range gradient issues are less severe, (ii) the smaller parameter count reducing overfitting risk, and (iii) the lower capacity acting as implicit regularisation.

VI. CHALLENGES FACED

Building a real-time market prediction system entailed several non-trivial engineering and modelling challenges, detailed below.

A. Data Quality and Alignment

Financial data obtained via public APIs contains irregular timestamps, corporate action adjustments (splits, dividends), and occasional missing bars on market holidays. Aligning daily OHLCV rows from five different tickers required careful date-based joining and NaN-handling to prevent inadvertent misalignment of the target label y_t.

B. Sentiment Date Assignment

The Kaggle financial-news corpus lacks per-sentence timestamps. Sentences were assigned random trading dates within the study window, which introduces stochastic noise into the sentiment signal. While this provides a distributional baseline, it weakens temporal alignment and likely reduces the marginal benefit of including the corpus. A production system would require a timestamped scrape or event-matched database.

C. Class Imbalance

The 52.8% / 47.2% Up/Down split is mild but sufficient to cause models to bias predictions toward the majority class. Class-weighted cross-entropy was employed as a mitigation strategy; however, the optimal weight schedule is hyper-parameter-sensitive and was not exhaustively tuned.

D. Look-Ahead Bias Prevention

Ensuring no future information leaks into training features required strict discipline: the Min-Max scaler was fitted exclusively on the training fold, the forward-looking close price Close_{t+1} was only used to construct the label y_t (never as an input feature), and rolling statistics were computed with `min_periods` constraints to avoid partial-window contamination.

E. RSS Feed Availability

Live RSS feeds are subject to rate limiting, URL deprecation, and geographic access restrictions. During development, several feed endpoints returned empty or malformed XML, requiring fallback logic and error-tolerant parsing.

F. Hyperparameter Sensitivity

A 27-run grid search was computationally tractable on a T4 GPU but remains coarse-grained. Learning rate, hidden dimension, number of layers, and dropout probability were fixed across architectures to maintain experimental control, yet these choices may not be individually optimal for each model. Bayesian optimisation over a continuous space would be a more thorough approach.

G. Generalisation Across Regimes

The study window (2022–2024) includes a bear market (2022 drawdown), a recovery (2023), and renewed growth. Models trained on a mixed-regime dataset may not generalise to a purely trending or volatile out-of-sample period. Walk-forward validation would better reflect real deployment performance.

H. Absence of Transaction Costs

The evaluation metric is raw directional accuracy. A model with 56% accuracy does not automatically yield a profitable strategy once bid-ask spreads, commission, slippage, and position-sizing constraints are factored in. Backtesting with realistic market-impact models remains as future work.

VII. Conclusion

This paper presented a comprehensive real-time market movement prediction system integrating technical indicators, multi-source news sentiment, and three deep recurrent architectures. The key findings are:

1) **Feature richness matters.** The combination of 15 technical features and 3 VADER-derived sentiment features consistently lifts accuracy above the 50% random baseline across all architectures.
2) **Simpler models can win.** The Vanilla RNN (18 K parameters) outperformed LSTM (209 K) and GRU (157 K) on both accuracy and F1 for 10-step financial sequences, consistent with the principle that model capacity should be matched to sequence complexity.
3) **Early stopping is essential.** In 24 of 27 grid runs, patience-5 early stopping prevented overfitting and identified optimal checkpoints substantially before the epoch budget was exhausted.
4) **Sentiment adds signal but needs timestamps.** The VADER pipeline is lightweight and domain-appropriate, but the random date assignment of the Kaggle corpus introduces noise that limits its practical contribution.

Future directions include: (i) replacing random date assignment with event-aligned sentence timestamps; (ii) extending to intra-day data; (iii) evaluating Transformer-based sequence models [11]; (iv) ablation studies isolating the contribution of each feature group; and (v) full-cycle backtesting with realistic transaction cost modelling.

References

[1] E. F. Fama, "Efficient capital markets: A review of theory and empirical work," *Journal of Finance*, vol. 25, no. 2, pp. 383–417, 1970.

[2] R. Choudhry and K. Garg, "A hybrid machine learning system for stock market forecasting," *World Acad. Sci. Eng. Technol.*, vol. 39, pp. 315–318, 2008.

[3] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[4] K. Cho, B. van Merrienboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, "Learning phrase representations using RNN encoder-decoder for statistical machine translation," in *Proc. EMNLP*, 2014, pp. 1724–1734.

[5] T. Fischer and C. Krauss, "Deep learning with long short-term memory networks for financial market predictions," *European Journal of Operational Research*, vol. 270, no. 2, pp. 654–669, 2018.

[6] C. Hutto and E. Gilbert, "VADER: A parsimonious rule-based model for sentiment analysis of social media text," in *Proc. ICWSM*, 2014.

[7] A. Malo, A. Sinha, P. Korhonen, J. Wallenius, and P. Takala, "Good debt or bad debt: Detecting semantic orientations in economic texts," *Journal of the Assoc. for Information Science and Technology*, vol. 65, no. 4, pp. 782–796, 2014.

[8] J. Bollen, H. Mao, and X. Zeng, "Twitter mood predicts the stock market," *Journal of Computational Science*, vol. 2, no. 1, pp. 1–8, 2011.

[9] X. Ding, Y. Zhang, T. Liu, and J. Duan, "Deep learning for event-driven stock prediction," in *Proc. IJCAI*, 2015, pp. 2327–2333.

[10] W. Liu, X. Zheng, S. Ye, and B. Shi, "Incorporating news sentiment into stock prediction: A deep learning approach," *Applied Soft Computing*, vol. 98, p. 106806, 2021.

[11] B. Lim, S. O. Arik, N. Loeff, and T. Pfister, "Temporal fusion transformers for interpretable multi-horizon time series forecasting," *International Journal of Forecasting*, vol. 37, no. 4, pp. 1748–1764, 2021.

[12] R. Aroussi, "yfinance: Download market data from Yahoo! Finance's API," [Online]. Available: https://pypi.org/project/yfinance/, 2019.

[13] Y. Nie, N. H. Nguyen, P. Sinthong, and J. Kalagnanam, "A time series is worth 64 words: Long-term forecasting with transformers," in *Proc. ICLR*, 2023.

[14] Z. Huang and J. Li, "A survey of deep learning in financial time-series forecasting," *IEEE Access*, vol. 8, pp. 203,741–203,756, 2020.